Assignment 2: Comparing Reinforcement Learning Algorithms

Submission rules: Python file only

Submission is individual only. This assignment must be submitted by each student separately. No pairs, no groups, and no shared implementation files.
Submit exactly one file: `assignment2_<student_id>.py`
Everything must be inside the Python file: code, experiments, plots generation, results summary, and written answers to the comparison questions. Do not submit a separate PDF report, notebook, ZIP file, Word file, images, folders, or external data files.

1. Assignment Definition

Objective. The goal of this assignment is to apply and compare different reinforcement learning (RL) algorithms on the same game or environment. You will implement and evaluate the following algorithms:
• Dynamic Programming
• Monte Carlo Methods
• Temporal Difference (TD) Learning
• Q-Learning
• SARSA
• TD(lambda) with Eligibility Traces
You will analyze their performance, strengths, and weaknesses in terms of convergence speed, computational complexity, and ability to handle delayed rewards.

Step 1: Choose a Game or Environment

Select a simple game or environment that can be modeled as a Markov Decision Process (MDP). Please ensure that your chosen environment is not the same as other students' choices to encourage creativity and diversity in problem-solving.
• Grid World: a grid-based environment where the agent navigates from a start state to a goal state while avoiding obstacles.
• Cart-Pole: a classic control problem where the agent must balance a pole on a cart.
• Frozen Lake: a slippery grid world where the agent must navigate from the start to the goal while avoiding holes.
• Custom Game: design your own small-scale game with well-defined states, actions, rewards, and transitions.
The game must have finite state and action spaces, a clear reward structure, and deterministic or stochastic transitions.

Step 2: Implement the RL Algorithms

• Dynamic Programming: use Value Iteration or Policy Iteration. Assume full knowledge of transition probabilities and rewards.
• Monte Carlo Methods: simulate episodes under a random or exploratory policy and estimate value by averaging returns.
• TD(0): update the value function after each step and compare with Monte Carlo.
• Q-Learning: estimate the optimal action-value function using an epsilon-greedy policy.
• SARSA: evaluate the current behavior policy and compare with Q-Learning.
• TD(lambda): extend TD learning with eligibility traces. Experiment with lambda values such as 0.2, 0.5, 0.8.

Step 3: Run Experiments

• Use comparable parameters across algorithms: gamma around 0.9, alpha around 0.1 when relevant, epsilon around 0.1 for epsilon-greedy methods, and enough episodes/steps for learning.
• Record convergence speed, computational cost, and policy quality such as average reward per episode.

• Generate visualizations from the Python file, such as value functions, Q-values, policy maps, and reward/convergence curves.

Step 4: Compare Results Inside the .py File

Your written answers must appear inside the Python file, for example in a top-level multiline string called REPORT or in a dictionary called WRITTEN_ANSWERS. Address:
• Convergence: which algorithm converges fastest and why; how gamma, alpha, and lambda affect convergence.
• Handling delayed rewards: which algorithm handles delayed rewards best; how TD(lambda) compares to TD(0) and Monte Carlo.
• Exploration vs. exploitation: differences between Q-Learning and SARSA; when SARSA may outperform Q-Learning.
• Computational complexity: which method is most efficient; memory/runtime impact of eligibility traces.
• Policy quality: which algorithm produces the best policy; when suboptimal but safer policies may be acceptable.

As we learned in class, code quality is part of this assignment. You are expected to write high-quality Python code according to the principles learned in the course.